

CS 156

Intro to AI

Heuristics, Local Search



These slides are based on the slides created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

The artwork is by Ketrina Yim

Today

- ▶ Properties of Heuristics
- ▶ Homework 2
- ▶ Local Search

Dominance

Dominance: $h_a \geq h_c$ if
 $\forall n \ h_a(n) \geq h_c(n)$

h_a
|
 h_c

If h_a is admissible, what does that tell us about h_c ?

Dominance

Dominance: $h_a \geq h_c$ if
 $\forall n \ h_a(n) \geq h_c(n)$

h_a
|
 h_c

If h_c is admissible, what does that tell us about h_a ?

Heuristics form a semi-lattice

Dominance: $h_a \geq h_c$ if

$$\forall n \ h_a(n) \geq h_c(n)$$

Heuristics form a semi-lattice:

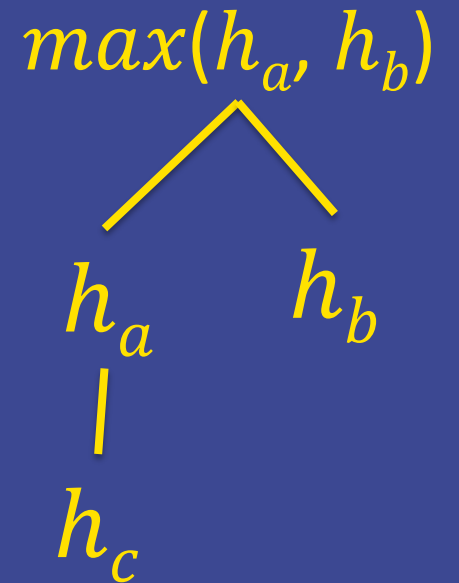
- Max of admissible heuristics:

$$h(n) = \max(h_a(n), h_b(n))$$

h_a and h_b are admissible. Is h admissible?

A. Yes

B. No



Heuristics form a semi-lattice

Dominance: $h_a \geq h_c$ if

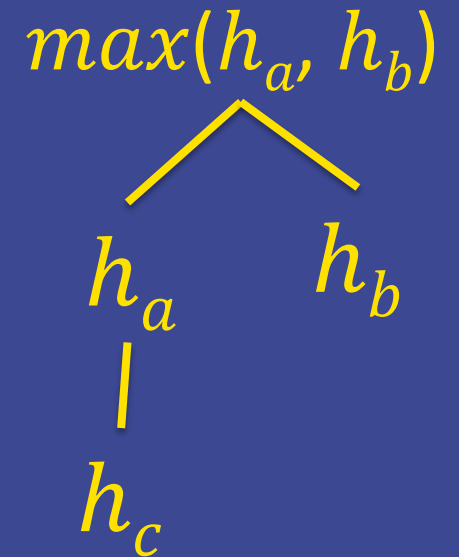
$$\forall n \ h_a(n) \geq h_c(n)$$

Heuristics form a semi-lattice:

- Max of admissible heuristics:

$$h(n) = \max(h_a(n), h_b(n))$$

Maximum of two or more admissible heuristics is admissible: **useful property for creating new, better heuristics.**



Trivial Heuristics

Dominance: $h_a \geq h_c$ if

$$\forall n \ h_a(n) \geq h_c(n)$$

Heuristics form a semi-lattice:

- ▶ Max of admissible heuristics:

$$h(n) = \max(h_a, h_b)$$

Trivial heuristics:

- ▶ Bottom of lattice is the null/zero heuristic (what does this give us?)
- ▶ Top of lattice is the exact heuristic



A* Applications

- ▶ Video games
- ▶ Pathing / routing problems
- ▶ Resource planning problems
- ▶ Robot motion planning
- ▶ Natural language processing, parsing
- ▶ Machine translation
- ▶ Speech recognition
- ▶ ...

Homework 2

- ▶ Question 1: Implement A*
- ▶ A* is similar to UCS but uses a different priority
- ▶ *astar* is a higher order function. It takes a function as an argument.
def *astar*(problem, **heuristic**):
- ▶ **heuristic** denotes a function that takes a state and a problem as arguments: *heuristic*(some_state, problem)
- ▶ heuristic could be: *null_heuristic*, *single_heuristic*, *better_heuristic*, *gen_heuristic* or any other heuristic you choose to add.

Homework 2

- ▶ Question 2: Implement *single_heuristic*, based on the **Manhattan distance**, that can be used when we know for sure that **there is only one medal in the quest**.
- ▶ Implement a *manhattan_distance* function that takes two locations and returns the Manhattan distance between them. *single_heuristic* should call *manhattan_distance* **if the state is not a goal state**.
- ▶ Remember: admissible heuristics represent **the actual cost (in carrots) of exact solutions to relaxed, easier problems**. What is the easier problem that the Manhattan distance is based on? There are two relaxations here.

Homework 2

- ▶ Question 3: Implement a better heuristic that can be used when we know for sure that **there is only one medal in the quest**.
- ▶ What relaxation can you reinstate from the previous question?
- ▶ Implement a helper function *carrots_to_medal* .
better_heuristic should call *carrots_to_medal*.

Homework 2

- ▶ Question 4: Implement a more general heuristic to be used when **the maze contains an arbitrary number of medals** and these medals can be anywhere in the maze.
- ▶ Remember: admissible heuristics represent **the actual cost (in carrots) of exact solutions to relaxed, easier problems**.
- ▶ One way to relax this complex problem is to assume that there is only one medal in the maze. For each medal you choose, you get a possible admissible heuristic. How would you then pick the best one? Think of a property of heuristics to use.

state vs problem

- ▶ All the heuristics are passed two arguments: *state* and *problem*.
- ▶ *state*: Sammy's current position and remaining medals.
- ▶ You must use the state passed and not `problem.start_state()`!
- ▶ *problem*:

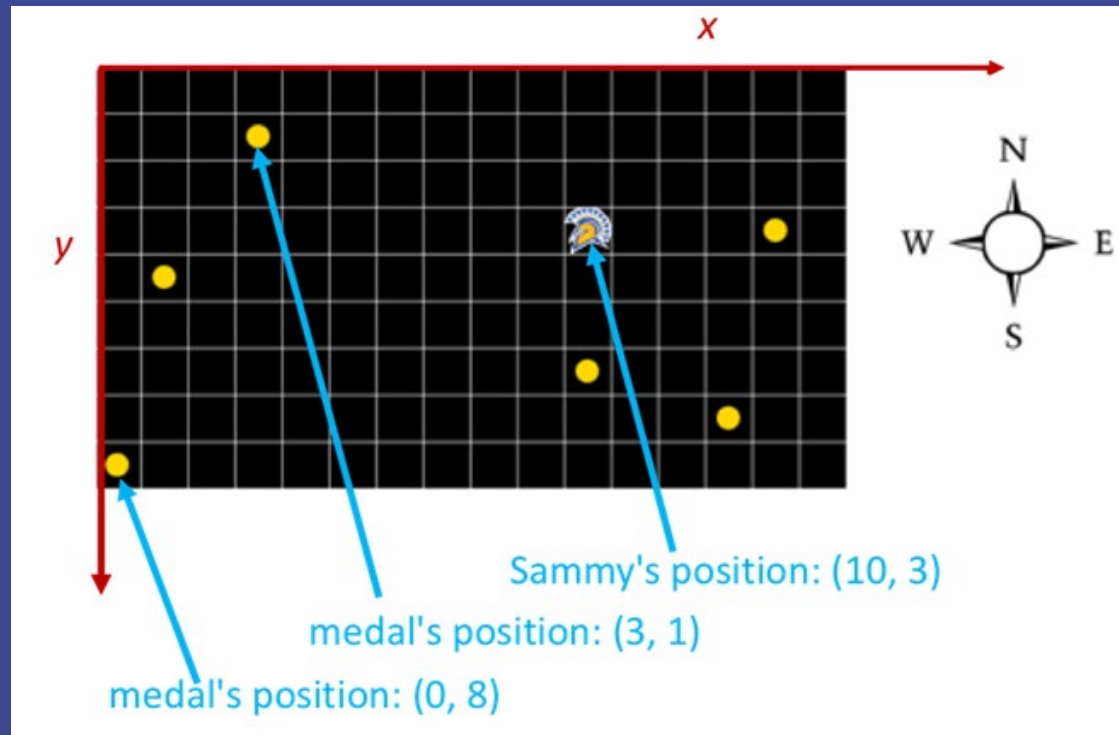
```
class Problem(object):  
    NORTH = "N"  
    SOUTH = "S"  
    EAST = "E"  
    WEST = "W"  
    # The cost (number of carrots consumed) associated with each move.  
    cost = {EAST: 15, WEST: 1, SOUTH: 2, NORTH: 14}  
  
    def is_goal(self, state):
```

state vs problem

- ▶ All the heuristics are passed two arguments: *state* and *problem*.
- ▶ *state*: Sammy's current position and remaining medals.
- ▶ You must use the state passed and not `problem.start_state()`!
- ▶ *problem*:
 - `problem.is_goal(state)`
 - `problem.cost` cost dictionary
 - `problem.NORTH`, `problem.SOUTH`, `problem.WEST`, `problem.EAST`
 - Example: `problem.cost[problem.NORTH]`

Grid Coordinates

- ▶ The top left position is (0, 0). The x coordinate increases as we move right in the grid and the y coordinate increases as we move down in the grid - as shown below.



Important!

- ▶ ALL the heuristics must return 0 when the state passed is a goal state.

Other Considerations

- ▶ The implementation is **memory and time efficient**: it does not store large data structures in memory and it does not perform unnecessary computations.
- ▶ **DRY code: Do not repeat the same code!** Reuse *carrots_to_medal* from Question 3 in Question 4.

Local Search



What is Search For?

Assumptions about the world:

- ▶ single agent, deterministic, fully observable, discrete

Search problems so far:

- ▶ **The path to the goal** is the solution
- ▶ Paths have various costs, depths
- ▶ Heuristics give problem-specific guidance
- ▶ Examples: pathing problems, Eight puzzle

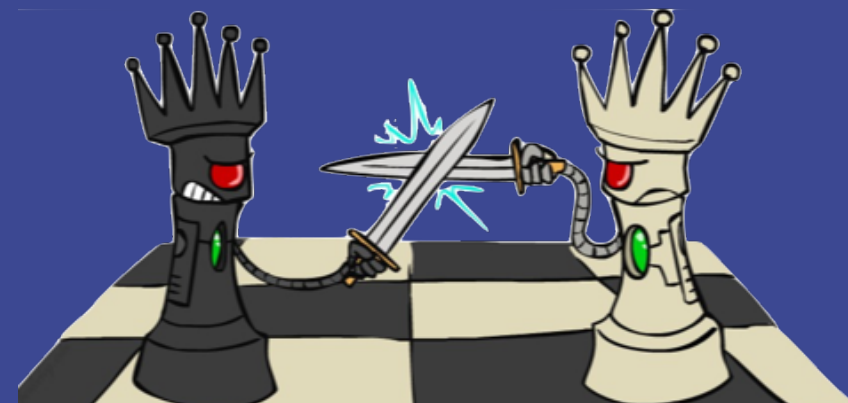
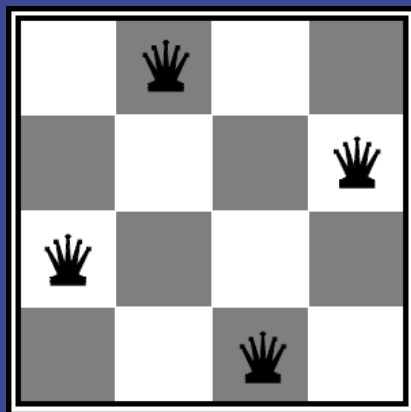
For some problems:

- ▶ **We need to find a goal state**
- ▶ It does not matter how we get there

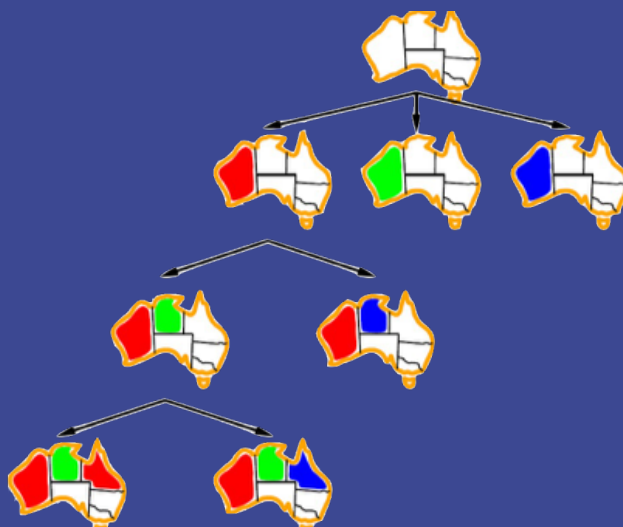


Different Search Problems

- ▶ Queens puzzle



- ▶ Coloring problem



Different Problems, Different Search

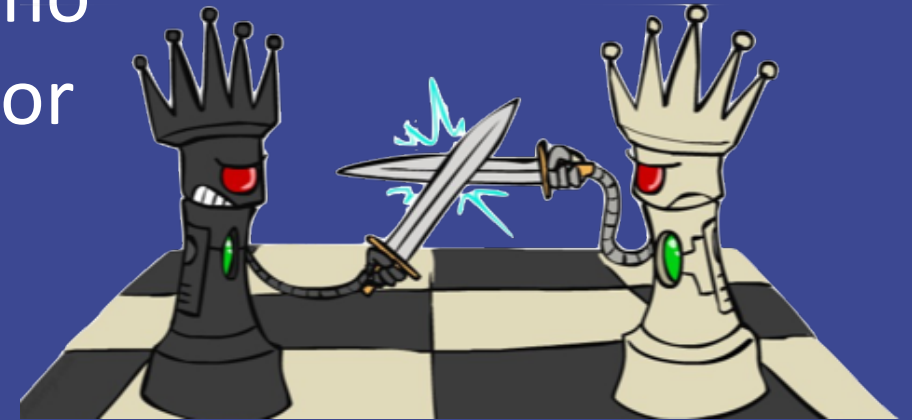
- ▶ Categories:
 - Optimization problems: find the best state given an objective function
 - Identification problems, scheduling problems, etc...
- ▶ We can **use iterative improvement algorithms** (local search):
 - Hill Climbing (steepest ascent / gradient descent)
 - Random restart hill climbing
 - Simulated annealing
 - Genetic Algorithms

Local Search

- ▶ The search algorithms we have seen so far keep unexplored alternatives on the fringe to ensure completeness.
- ▶ Local search:
 - keep a single 'current' node (no fringe)
 - try to improve it until we can't make it better
- ▶ Advantages:
 - Local search is generally much faster and more memory efficient
- ▶ Disadvantages:
 - incomplete and suboptimal

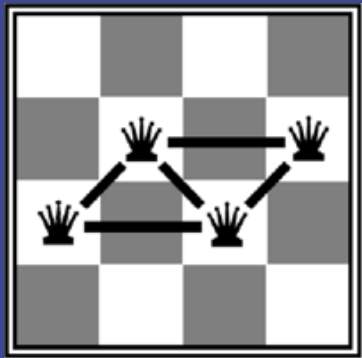
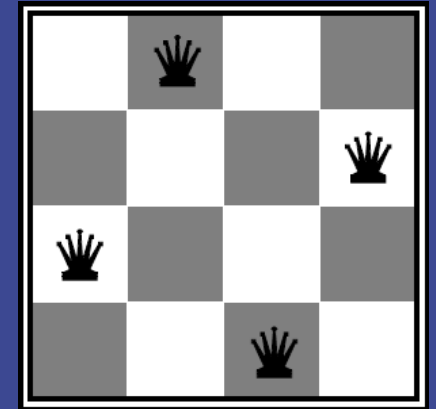
Example: n-Queens

- ▶ Place n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal.
- ▶ Start with any configuration.
- ▶ Move a queen to reduce the number of conflicts.

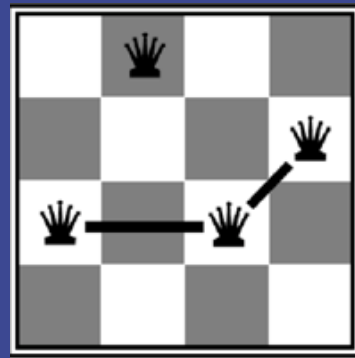
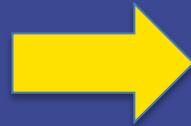


Example: 4-Queens

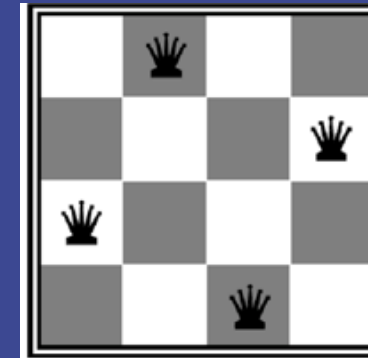
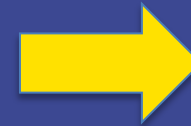
- ▶ States: 4 queens in 4 columns ($4^4 = 256$ states)
- ▶ Actions: move queen in column
- ▶ Goal test: no conflict
- ▶ Evaluation: $c(n)$ = number of conflicts



$c = 5$



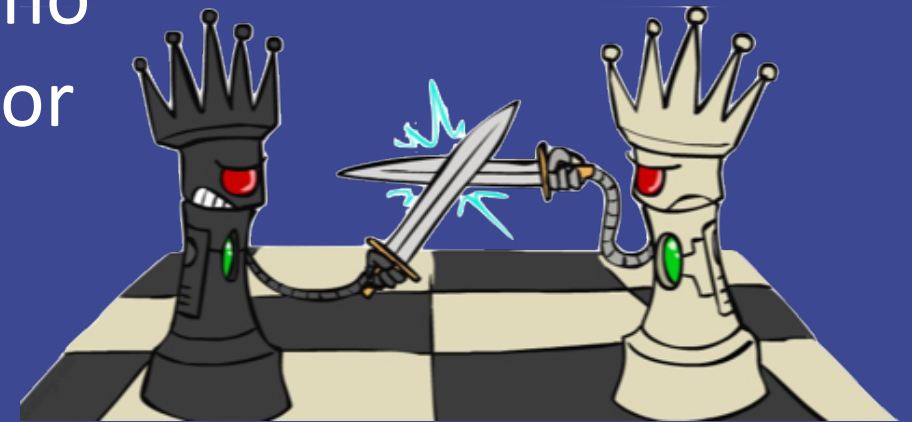
$c = 2$



$c = 0$

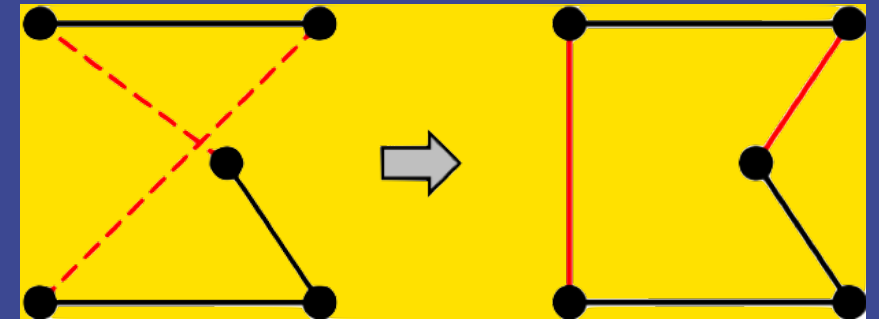
Example: n-Queens

- ▶ Place n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal.
- ▶ Start with any configuration.
- ▶ Move a queen to reduce the number of conflicts.
- ▶ Solve n-queens problems almost instantaneously for very large n (1 million)



Example: Travelling SalesPerson Problem

- ▶ Given a list of cities and the distances between each pair of cities, what is the **shortest** possible route that visits each city exactly once and returns to the origin city? It is an **NP-hard problem**.
- ▶ Start with any complete tour, perform pairwise exchanges.
- ▶ Variants of this approach get within 1% of optimal very quickly with thousands of cities



Hill Climbing (gradient ascent/descent)

'Like climbing Everest in thick fog with amnesia'

Simple, general idea:

- ▶ Start wherever
- ▶ Repeat: move to **the best neighboring state**
- ▶ If no neighbors better than current, quit

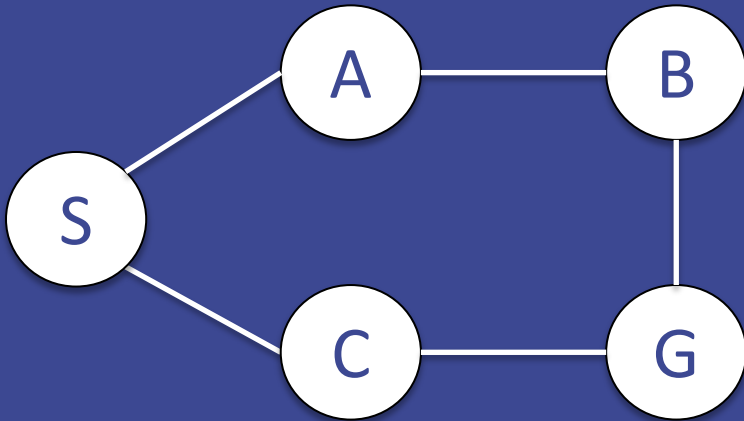


Hill Climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                  neighbor, a node

  current ← MAKE-NODE(INITIAL-STATE[problem])
  loop do
    neighbor ← a highest-valued successor of current
    if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
    current ← neighbor
  end
```

iClicker



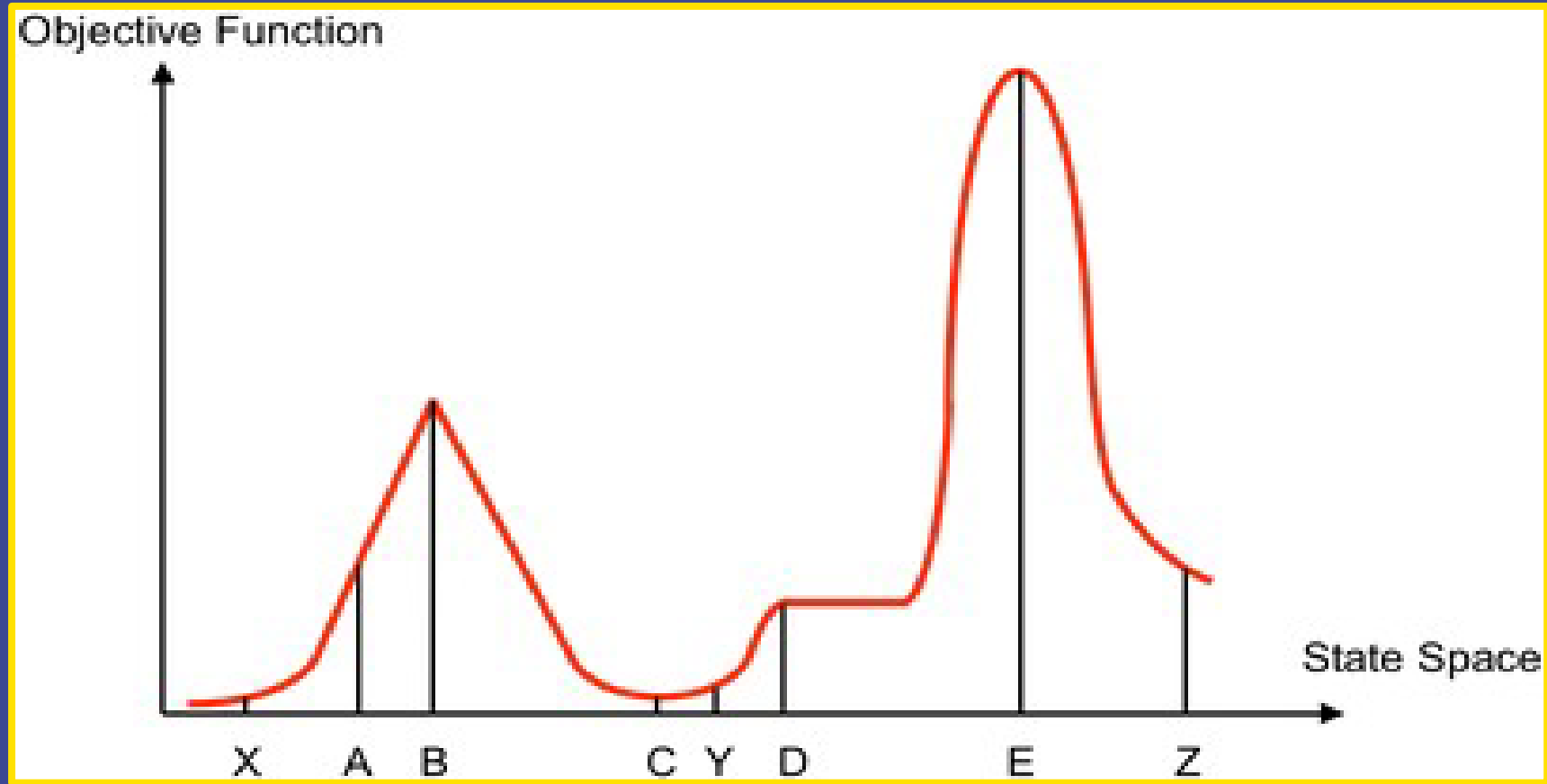
Objective Function

S	0
A	1
B	0
C	0
G	3

Starting from state S, we run the hill climbing algorithm to completion. Which state is returned?

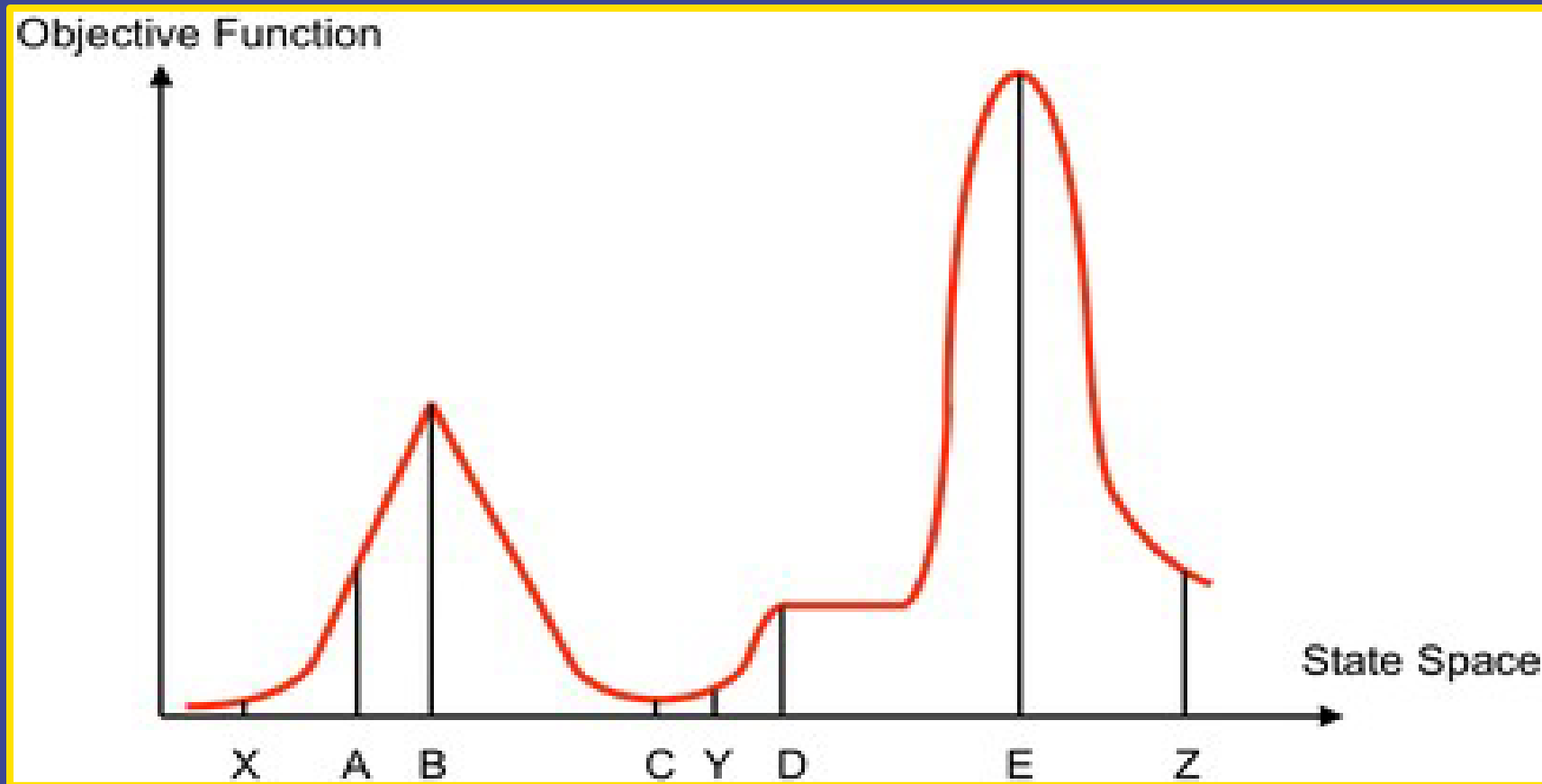
- A. A
- B. B
- C. C
- D. G
- E. S

Hill Climbing



One-dimensional state-space landscape

Hill Climbing

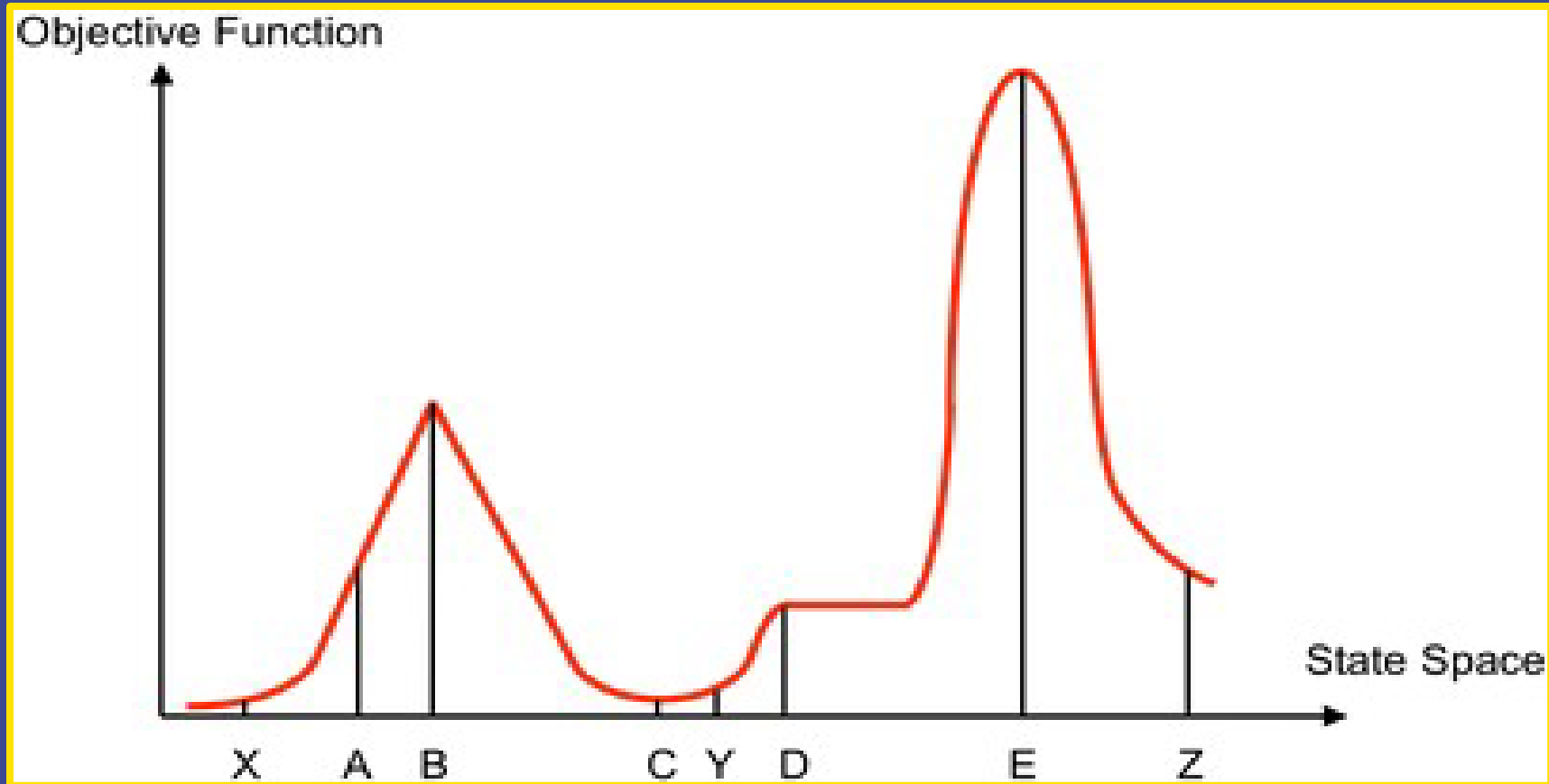


One-dimensional state-space landscape

Starting from state X, we run the hill climbing algorithm to completion. Which state is returned?

- A. A
- B. B
- C. C
- D. D
- E. E

Hill Climbing

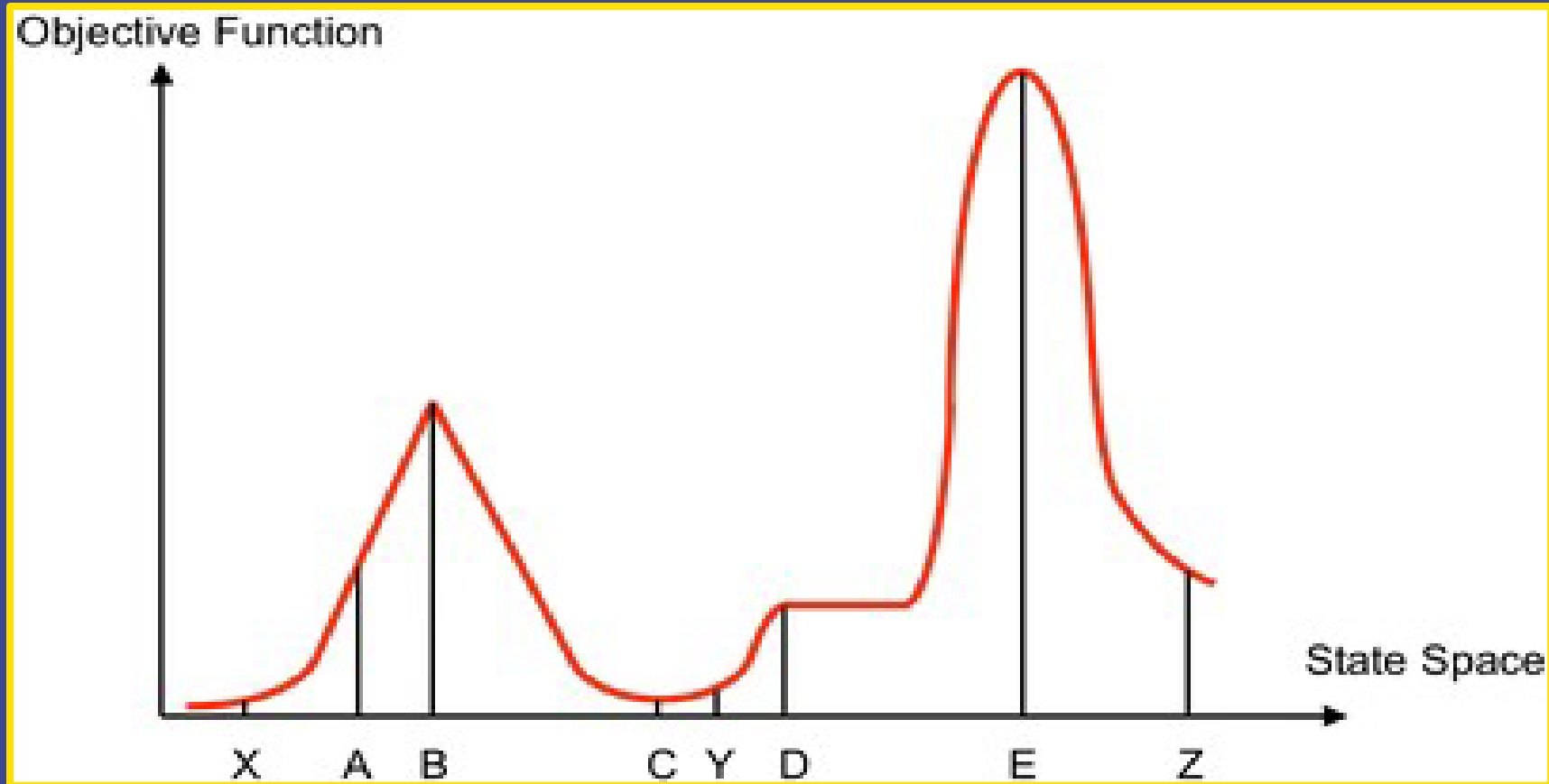


One-dimensional state-space landscape

Starting from state Y, we run the hill climbing algorithm to completion. Which state is returned?

- A. A
- B. B
- C. C
- D. D
- E. E

Hill Climbing

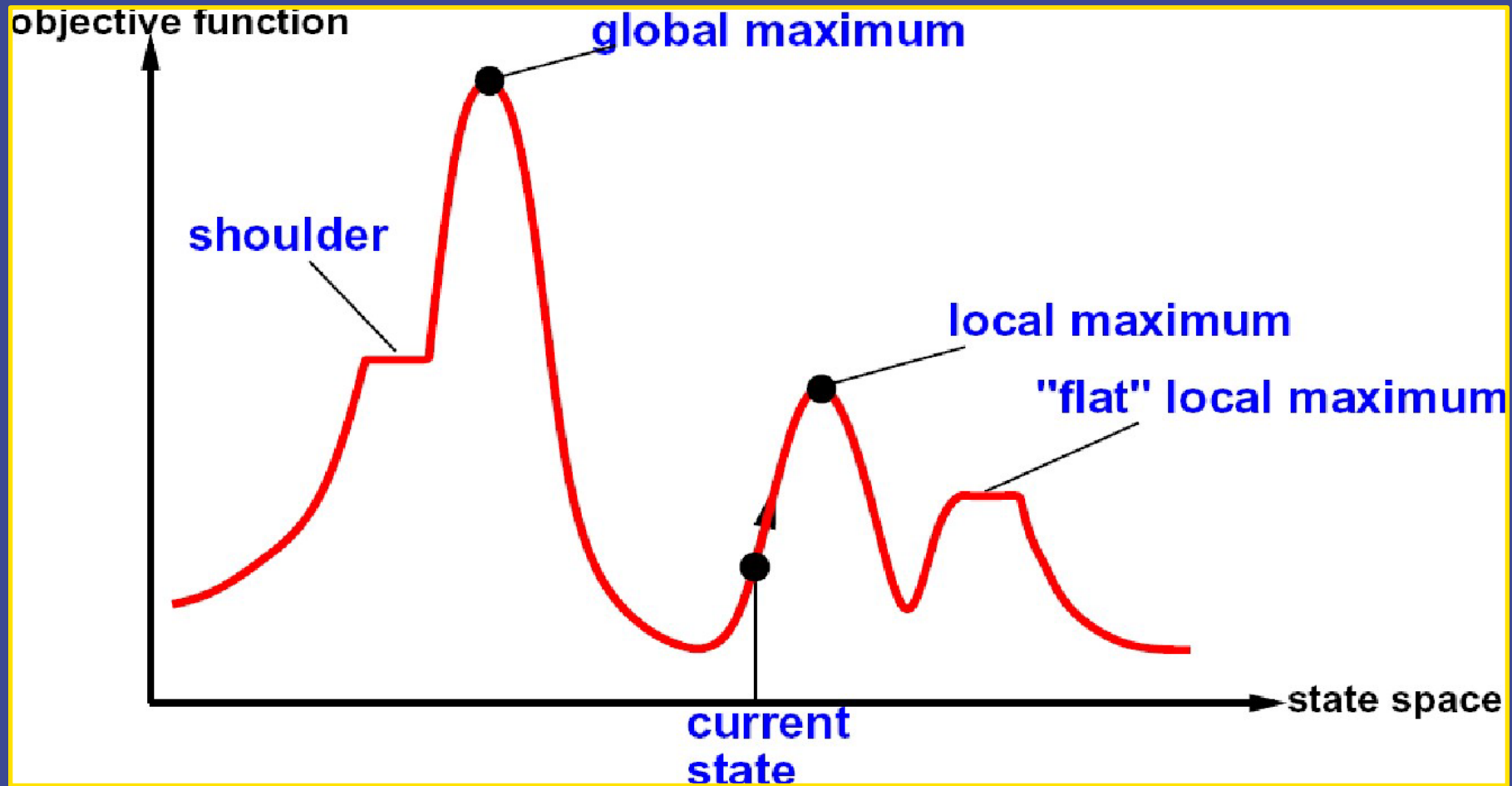


One-dimensional state-space landscape

Starting from state Z, we run the hill climbing algorithm to completion. Which state is returned?

- A. A
- B. B
- C. C
- D. D
- E. E

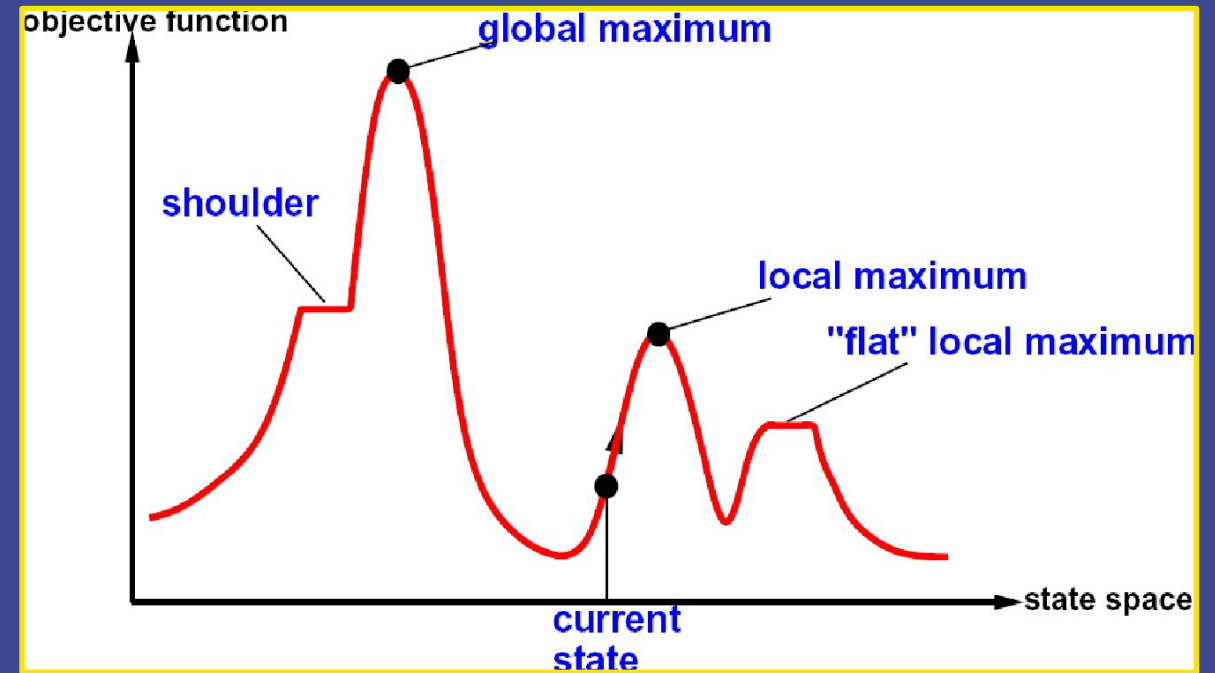
Hill Climbing



One-dimensional state-space landscape

Hill Climbing

- ▶ Complete?
- ▶ Optimal?
- ▶ Advantage?
 - For some problems, a non optimal solution is ok
- ▶ Can we make it better?



Random-restart Hill Climbing

- ▶ Idea: Overcome local maxima by conducting several hill-climbing searches, starting from a random initial state.
- ▶ We keep the best result.
- ▶ This is a surprisingly effective algorithm in many cases.

Simulated Annealing

Idea:

- ▶ Escape local maxima by allowing random downhill moves
- ▶ But make them rarer as time goes on

Simulated Annealing

function **SIMULATED-ANNEALING**(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

local variables: *current*, a node

next, a node

T, a “temperature” controlling prob. of downward steps

current $\leftarrow$ MAKE-NODE(INITIAL-STATE[*problem*])

for *t* $\leftarrow$ 1 **to** ∞ **do**

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow$ VALUE[*next*] – VALUE[*current*]

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

If *T* decreased slowly enough, it will converge to optimal state.

Simulated Annealing

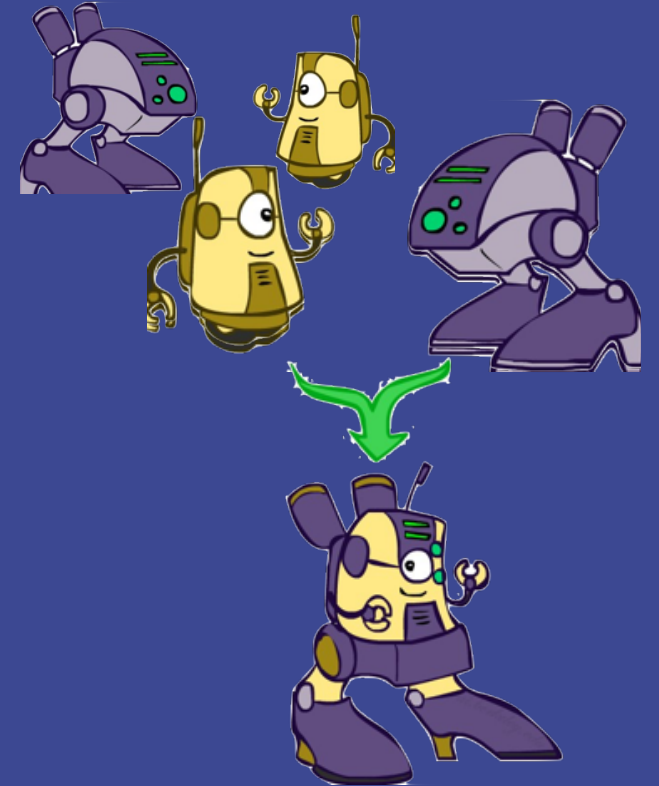
- ▶ Escape local maxima by allowing random downhill moves
- ▶ But make them rarer as time goes on



- ▶ The more downhill steps we need to escape a local optimum, the less likely we are to ever make them all in a row.

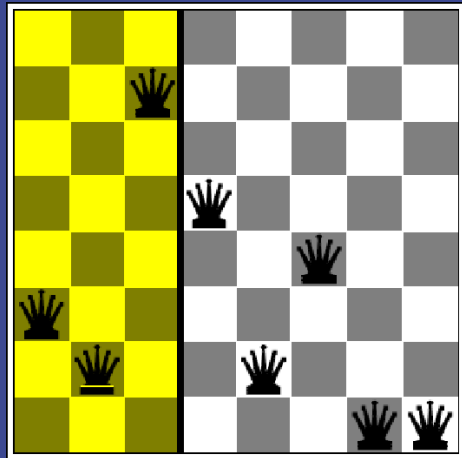
Genetic Algorithms

- ▶ Genetic algorithms use a natural selection metaphor
 - Start with a population of hypotheses (candidate solutions)
 - Keep best N hypotheses at each step (**selection**) based on a **fitness** function
 - Use pairwise **crossover** operators, with optional **mutation** to give variety



Genetic Algorithms

- ▶ Start with a randomly generated states: **population**
- ▶ Each state (**individual**) is represented as a string

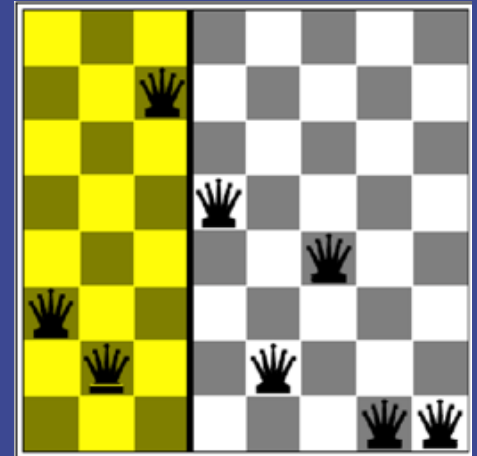


32752411

Genetic Algorithms

- ▶ Start with a randomly generated states: **population**
- ▶ Each state (**individual**) is represented as a string
- ▶ Each state is **rated** by an objective function (**fitness function**)
 - number of conflicts?
 - number of non-attacking pairs of queens
= total pairs – conflicts

How many
conflicts in this
state?

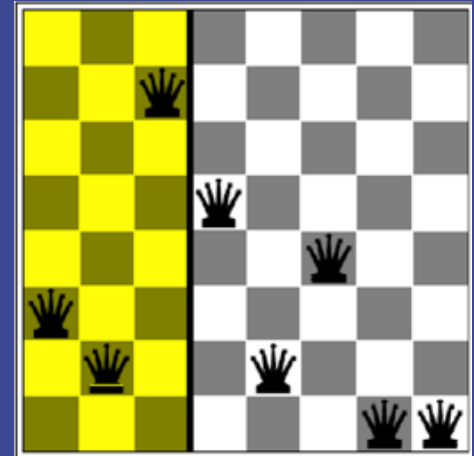


Genetic Algorithms

- ▶ Start with a randomly generated states: **population**
- ▶ Each state (**individual**) is represented as a string
- ▶ Each state is **rated** by an objective function (**fitness function**)
 - number of conflicts?
 - number of non-attacking pairs of queens

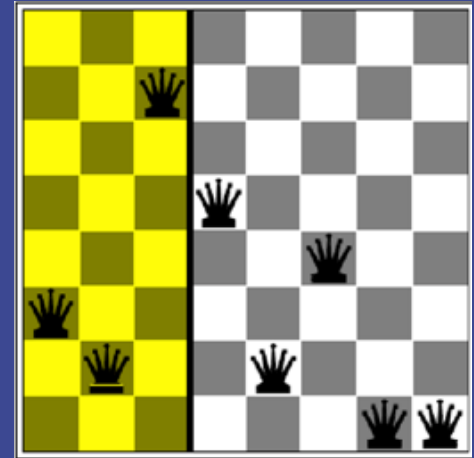
- = total pairs – conflicts

How many pairs of queens?



Genetic Algorithms

- ▶ Start with a randomly generated states: **population**
- ▶ Each state (**individual**) is represented as a string
- ▶ Each state is **rated** by an objective function (**fitness function**)
 - number of conflicts?
 - number of non-attacking pairs of queens
= total pairs – conflicts
= $8 C 2 - 5$
= $7 + 6 + 5 + 4 + 3 + 2 + 1 - 5$
= 23

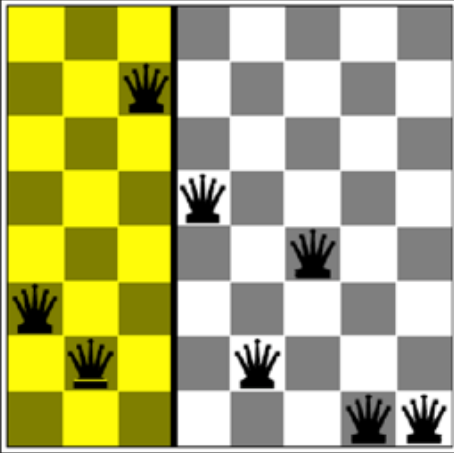


$f = 23$

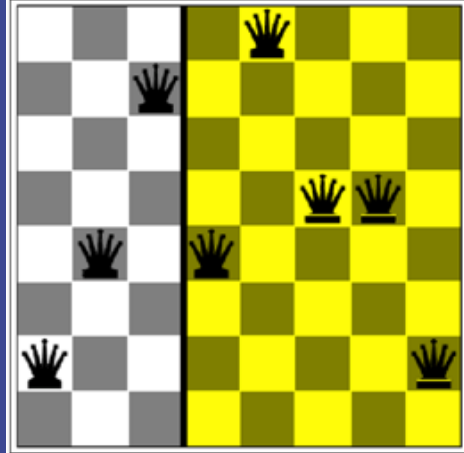
Genetic Algorithms

- ▶ Start with a randomly generated states: **population**
- ▶ Each state (**individual**) is represented as a string
- ▶ Each state is **rated** by an objective function (**fitness function**)
- ▶ States are chosen based on **probability proportional to fitness function**
- ▶ Use pairwise **crossover** operators
- ▶ Add **mutation** to give variety

Genetic Algorithms: N-Queens

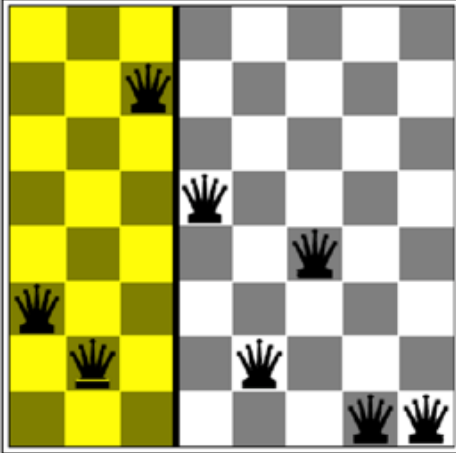


32752411



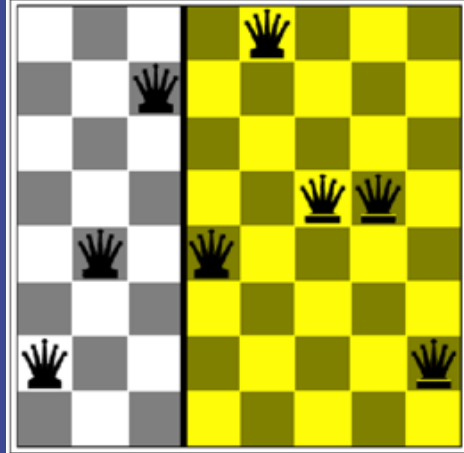
iClicker:
How would we represent
this state?

Crossover: N-Queens



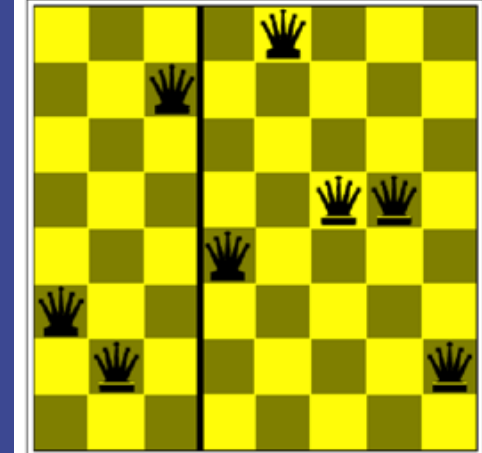
32752411

+



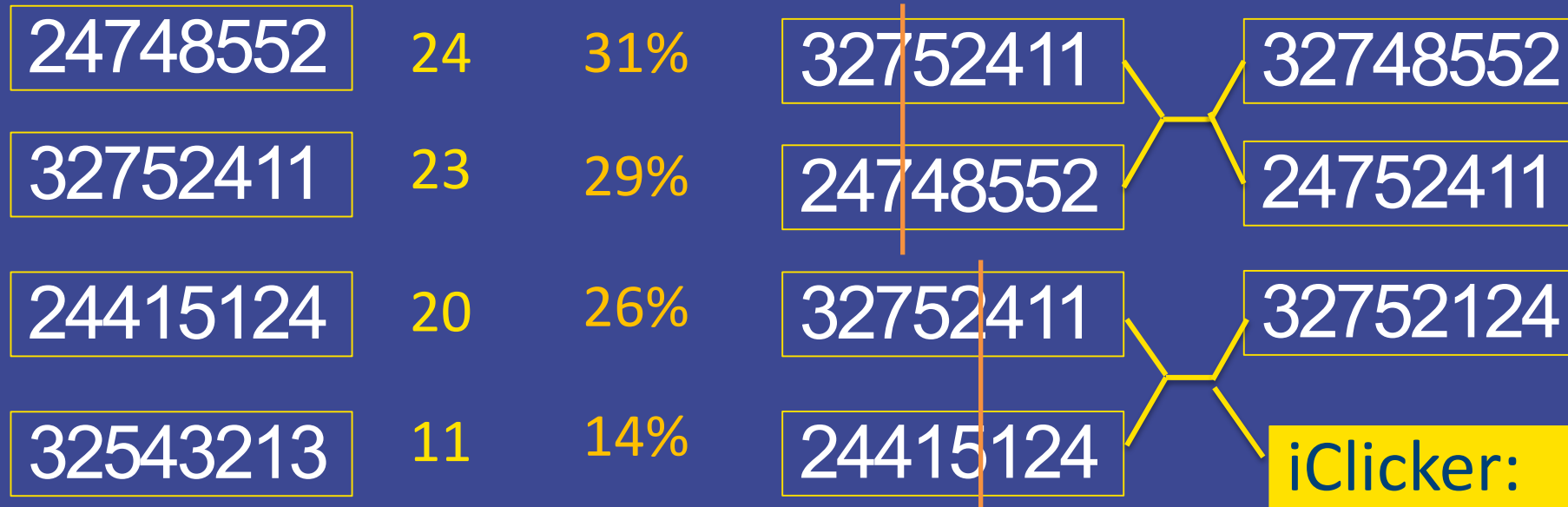
24748552

=



32748552

Genetic Algorithms



- Fitness: non-attacking pairs of queens
- Probability : proportional to fitness

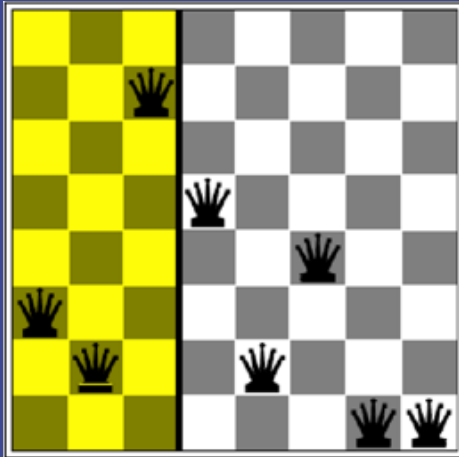
iClicker:
What is the value
generated here?

Genetic Algorithms



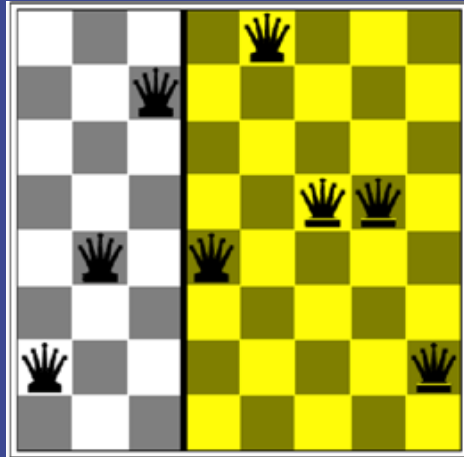
- Fitness: non-attacking pairs of queens
- Probability : proportional to fitness
- Mutation

Genetic Algorithms: N-Queens



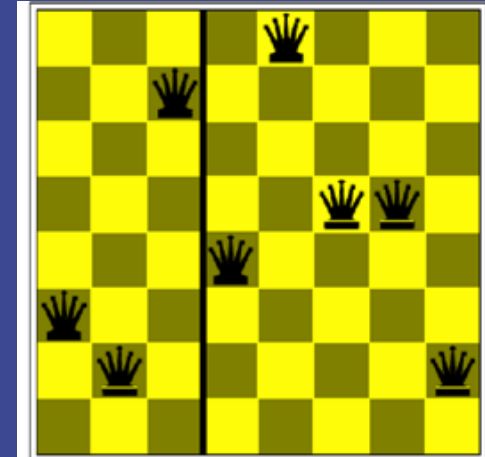
32752411

+



24748552

=



32748552

- Why does crossover make sense here?
- When wouldn't it make sense?

Genetic Algorithms

► Applications

- Bioinformatics
- Automated Design
- Artificial Creativity
- Feature Selection in Machine Learning
- Malware Classification Algorithms
- ...

To Do

- ▶ Question of the week on Tuesday
- ▶ Homework 2
 - Due Sep 23 at 5 PM
 - Questions? Canvas discussion forum